Panorama Season 2 Auction

Security Review Report (Public Edition)

Field	Value
Publisher	Trislit Consulting LLC
Date	15 July 2026
Engagement	Security review of Season 2 multi-unit uniform-price auction
Scope	Smart contracts (PanoramaSeason2Auction and mint trust surface), settlement scripts, Next.js frontend
Out of scope	Economic/MEV strategy advice; off-chain monitor scripts; formal verification of the full EVM
Method	Manual review (two independent passes); Foundry suite (116 tests); differential clearing oracle (1,000 fuzz runs); liability-identity checks; mainnet fork validation; frontend adversarial review; cross-check against contracts/AUCTION_RUNBOOK.md
Classification	Public. Published with the Panorama team's permission ahead of the Season 2 launch.

About this document

This is the public edition of the review I delivered to the Panorama team. It is substantially the same document; findings were shared with the team first, and where something has already been fixed or confirmed as intended, that status is noted inline. A security review is a point-in-time exercise: it reflects the code and on-chain state as of the dates above, and it is not a guarantee that no defect exists.

One thing worth saying to bidders directly: nothing in this review found a way for an outside attacker to take escrowed ETH. The findings below are about operational discipline and frontend behavior, and the team was already ahead of most of them in their own runbook.

---

1. Executive Summary

1.1 Purpose

The question that matters for launch is simple: is bidder ETH safe in escrow, and does the published operations path hold up? Everything in this report works backward from that.

1.2 Summary

Escrow accounting holds up. I didn't find a path that lets an unprivileged attacker drain bidder ETH, double-spend refunds, or leave the contract unable to pay legitimate claimants. I made two independent passes at this: the first was a full manual review plus the test work below, and the second traced the ETH conservation identity by hand across every code path and re-ran the suite. Both passes agree.

On operations and trust boundaries, I largely agree with the project's own AUCTION_RUNBOOK.md. Atomic Safe settlement, exact winner cap, pause behavior (the clock keeps running), cancel policy, batch sizing, and recovery backstops are already documented there. I confirmed those positions independently, including on a mainnet fork, so I won't restate them at length.

The one new contract finding is a griefing risk, not a fund-loss bug. If the sale hasn't been settled within 7 days of the auction ending, any wallet can cancel it and force full refunds (C-M1). Nobody loses principal, but the sale and the winners' allocation are gone. The fix is an operational deadline, not a contract change. The team has confirmed this permissionless

recovery is deliberate, a failsafe so bidders never depend on the owner to recover their ETH, and I agree with that tradeoff.

Beyond that, the new material is mostly frontend: production demo confusion, bidding while state is stale or degraded, and the demo environment flag short-circuiting production build gates. These are targeted fixes, not a rewrite.

1.3 Bottom line

Question	Answer
Can a random attacker steal escrowed ETH via a logic bug?	I found no evidence of one, on either pass
Can a random attacker cancel a successful sale?	Yes, but only after the 7-day grace, and it only refunds (no theft). Settle in time (C-M1)
Does the runbook settle path close the privileged mint window?	Yes. Use SettleAuctionAtomic.safeBatch(); never AuthorizeAuction for production
Are cancel/pause risks new discoveries?	No. Already in the runbook, and I agree with it (pause does not freeze the clock)
Is the frontend safe enough to ship?	Yes, with targeted fixes: block prod demo, gate stale bids, keep demo out of production builds
Rewrite required before launch?	No

1.4 Severity snapshot

Severity	Count	Theme
Critical	0	None found
High	3	Frontend: demo / stale writes / demo env gates (new relative to runbook)
Medium	7	One contract griefing/lost-sale item (C-M1); frontend wallet / RPC / config
Low / Informational	10	Contract notes (C-I1 to C-I4, incl. the unverified renderer rotation); gas/ops and polish (several already in runbook)
Agreed / documented	Several	Runbook items independently confirmed (Section 2)

1.5 Recommendations for launch

Follow the runbook (agreed):

1. Settle only via Safe SettleAuctionAtomic.safeBatch() after endTime, and within 7 days of the final endTime. Past that grace, anyone can cancel a successful sale (C-M1).
2. Do not use AuthorizeAuction or split finalize as the production settle path.
3. Keep pause/cancel policy as written (pause does not stop the clock; disclose if used).
4. Keep finalize batches at or below 45.

Add (frontend; not in the runbook):

5. Disable demo query flag on production hosts; show DemoBar on /recovery when demo is active.
6. Gate bidding writes when state is stale or degraded (same as admin/recovery).
7. Ensure NEXT_PUBLIC_AUCTION_DEMO cannot ship in production builds.

Should do (small frontend / copy):

8. Bind pending-transaction UI lock to the connected wallet.

9. Align WalletConnect metadata URL and block explorer with the real host/chain.

1.6 Verification beyond the runbook

- 116/116 auction Foundry tests passed (unit, invariant, differential, and liability-identity; re-run on the second pass).
- Differential clearing oracle: 1,000/1,000 randomized auction scenarios matched an independent reference model on clearing price, minimum bid, escrow sum, and reveal order.
- Mainnet fork: the privileged generation-path mint during a non-atomic gap is real, and exact-cap atomic settle leaves no leftover inventory. Both support the runbook preference for safeBatch().
- By hand: I traced the solvency identity (balance equals escrow plus pending returns plus unreleased proceeds) through every mutating function, including displacement, partial finalize, and every recovery path.
- Live mainnet check (15 July): the cap is still exhausted (totalMinted and mintCap both 90). The renderer was rotated to a new, unverified contract on 13-14 July; a bytecode scan shows no mint path as compiled (C-I4). I re-pinned the fork test to the new address and all 5 fork tests pass against current mainnet.

---

2. Agreement with AUCTION_RUNBOOK.md

The project already documented the operations and trust controls that matter. I checked each one independently and agree with them, so they're listed here as confirmations rather than repeated as findings.

Runbook item	Section	Position
Prefer Safe ownership; settle as one Safe safeBatch()	1, 5	Agree. Production path.
Exact winner count for Season 2 cap; no leftover headroom	5	Agree. Confirmed on fork.
AuthorizeAuction then separate finalize is legacy / not preferred for production	5	Agree. Do not use for this sale.
Pause stops bids but the clock keeps running; disclose; unpause with time or cancel	4	Agree. Keep bidder-facing copy aligned if used.
Cancel only for security or broken sale, not price reaction	4	Agree.
Keep finalize batches at or below 45 (gas)	5	Agree.
Supply-mismatch and minting-unavailable recovery paths	6	Agree. Intended backstops; settle within grace.
No collection mutations during live bidding	4	Agree.

One note worth keeping straight (this is not a new High finding): a random outside wallet cannot mint Season 2 IDs. mintTo is privileged (onlyOwnerOrOperator), and the runbook's atomic settle closes the privileged window. The generation-pipeline roles need mint access by design; that's a feature of the system, not a stray permission.

---

3. System Overview

The trust model matches the runbook: the auction owner can't sweep escrow as long as the accounting holds, but NFT allocation still depends on settlement hygiene and the privileged mint roles. That split (funds safe by code, allocation safe by procedure) is the frame for everything below.

---

4. Methodology

Pass	Focus	Outcome
0	Cross-check AUCTION_RUNBOOK.md	Ops/trust items largely pre-documented; I agree with them
1	Threat model, privileged paths, settlement scripts	Aligns with runbook; no new Critical contract logic
2	Fund-loss logic hunt	No unprivileged drain found at any severity
3	Differential oracle, liability identity, mainnet fork	All passed; supports atomic exact-cap settle
4	Second independent pass: hand-trace of ETH conservation, heap correctness, recovery interactions	Confirms passes 1-3; surfaced C-M1 and the informational notes
Frontend	Write path, demo, stale state, env, CSP	3 High frontend issues (Section 5)

---

5. Findings

Severity definitions used in this report:

Severity	Definition
Critical	Unprivileged theft or permanent loss of user principal; no designed recovery
High	Material harm or severe user deception under realistic conditions
Medium	Significant disruption or fairness issue; often procedure-mitigated
Low	Limited impact, intentional tradeoff, or polish
Agreed	Already covered in project runbook; independently confirmed

5.1 High (frontend)

F-H1: Production demo via demo query parameter; recovery lacks DemoBar

Field	Value
Severity	High
Location	useAuctionSession.ts; /recovery has no DemoBar
Confidence	High

Description: On a live host, the demo query parameter switches the session to an in-memory auction. Main and admin show a DemoBar; /recovery does not. A user on the recovery page can believe they withdrew or recovered funds when nothing hit the chain. This is not a fund drain. It is a deception surface, and recovery is the worst page to have it on.

Recommendation: Ignore the demo query parameter in production builds. Always show DemoBar (or an equivalent watermark) on /, /admin, and /recovery whenever demo is active.

Status: The team has confirmed the demo flag will be removed before production.

F-H2: Bidding page ignores stale / degraded write locks

Field	Value
Severity	High
Location	page.tsx (writeBlocked = busy only); admin/recovery already gate
Confidence	High

Description: The primary bid/raise/withdraw rail does not lock on stale or degraded snapshots, even though admin and recovery already do. So the pages the team uses fail closed under RPC stress, but the page bidders use does not. That inversion is the finding.

Recommendation: Set writeBlocked = busy \|\| degraded \|\| stale and show the same banner used on admin/recovery.

F-H3: NEXT_PUBLIC_AUCTION_DEMO disables production environment hard gates

Field	Value
Severity	High
Location	env.ts assertProductionEnv short-circuit on demo
Confidence	High

Description: A production build with the demo flag set can skip required RPC, WalletConnect, and deploy-block checks and ship a fake auction on a real domain.

Recommendation: Reject the demo flag entirely when the platform is production. Add checklist or CI coverage that the flag is unset.

Status: The team has confirmed the demo flag will be removed before production.

5.2 Medium (frontend)

ID	Finding	Recommendation
F-M1	/admin gate is client-only; onlyOwner on-chain is the real gate	Document; acceptable
F-M2	Pending-tx hydrate not bound to connected account (wallet B locked by A's unresolved bid; success UI can fire on wrong session)	Require tracker.account equals connected address on hydrate
F-M3	Explorer / CDN / GitHub URLs not cross-checked vs chain ID	Build-time asserts / checklist
F-M4	Broad public RPC fallbacks amplify stale reads (feeds F-H2)	Prefer primary RPC; hard degraded banner on fallback
F-M5	WalletConnect metadata URL fixed to panorama.garden	Set to actual Season 2 origin
F-M6	CDN metadata date into hero image URL	Defer; sanitize if easy

5.3 Medium (contract)

C-M1: A healthy sale can be cancelled by anyone once the 7-day finalize grace passes

Field	Value
Severity	Medium (griefing / lost sale, not lost principal)
Location	recoverFromMintingUnavailable; interacts with SettleAuctionAtomic.safeBatch()
Confidence	High

Description: The recommended settle path does not authorize the auction to mint until the settle transaction runs. That is deliberate and it is what closes the privileged mint window. The side effect is that mintingUnavailable() stays true for the whole gap between endTime and settlement. Once the clock passes endTime + FINALIZE_GRACE (7 days), any wallet can call recoverFromMintingUnavailable and cancel a fully successful, oversubscribed auction, refunding every winner. finalize cannot be used to defend the sale, because it reverts with NotOperatorAuthorized for the same reason the recovery is open. After 7 days the contract has no way to tell an owner who is slow to settle from one who never will, so it opens the recovery path for both.

Impact: No bidder loses principal. Everyone is refunded in full. What is lost is the sale itself and the winners' allocation. This is an availability and griefing issue, not theft.

Recommendation: Treat "settle within 7 days of the final endTime" as a hard operational deadline, not a guideline. Do not try to defend by pre-authorizing the operator early, because that reopens the mint window the atomic path is built to close. If on-chain protection is wanted in a future version, give the minting-unavailable recovery a longer grace than finalize so the honest settle window and the griefing window are not the same length.

Status: The team has confirmed the permissionless recovery is intended behavior, added deliberately as a failsafe so bidders never depend on the owner to get their ETH back. I agree with that design goal; this finding is simply the flip side of it, and the mitigation is the team settling promptly, which is already their plan. For bidders, the worst case here is a full refund, not a loss.

5.4 Informational (contract)

These are notes, not defects. They did not change the fund-safety result.

ID	Note
C-I1	cancelAuction is onlyOwner but not nonReentrant, unlike every sibling recovery function. It is safe here because effects come before interactions and _releaseProceeds is a no-op when there are no live bids. The inconsistency is worth a comment.
C-I2	finalize selects winners by rescanning the frozen set on each mint, so it is O(winners squared). Bounded and fine at 90 units with batches of 45 or less. This is the reason batch discipline is load-bearing, not just a gas nicety.
C-I3	mintCap() is a global sum across seasons, not per-season. The exact-cap story depends on the settle script setting the Season 2 cap to exactly winnerCount. The script does this and re-checks post-state, so it holds, but it is an off-chain guarantee rather than an on-chain one.
C-I4	The live renderer was rotated on mainnet on 13-14 July 2026 (setRenderer by the NFT owner) to 0xbd6D...89E3. At the time of review its source was not verified on Etherscan. The renderer holds latent mint privilege through onlyOwnerOrOperator, so this is a change to the mint trust surface, not just the art. I scanned the deployed bytecode: no mintTo selector is present, so it cannot mint as compiled, and the exhausted cap contains it regardless. Resolved: the team verified the contract source on Etherscan on 15 July, same day it was raised. I also re-pinned the fork test to the new address; all 5 fork tests pass against current mainnet.

5.5 Agreed runbook items (abbreviated)

These match AUCTION_RUNBOOK.md. Short notes only; follow the runbook.

Topic	Note
Atomic settle vs legacy authorize	Prefer SettleAuctionAtomic.safeBatch(). Do not use AuthorizeAuction for production. Privileged-only mint window if the wrong path is used; atomic exact-cap settle removes it.
Cancel / pause	Owner powers by design. Pause does not freeze the clock. Runbook policy is sound; keep bidder-facing disclosure aligned.
Finalize gas / batches ≤ 45	Already in runbook. Ops constraint, not fund drain.
Recovery graces	Intended backstops; settle within documented windows.
Force vs try ETH send	Intentional asymmetry; /recovery withdraw covers failed pushes.

---

6. Fund-Loss Paths Ruled Out

I looked for each of these specifically and did not find any of them:

- Unprivileged drain of totalEscrowed
- Double refund, or refund-plus-mint of the same bid (each bid is deleted before any external call, and refund batches skip already-processed entries)
- Heap corruption reachable through the public API that mis-assigns winners or mismatches escrow
- withdraw insolvency (a pending-returns ledger the balance cannot cover)
- ETH accepted with neither an NFT nor a refund/withdraw path in any designed terminal state

The liability identity (balance equals escrow plus pending returns plus unreleased proceeds) held under differential fuzzing, under the existing invariant suite, and under my own hand-trace of every mutating function. Forced donations only ever create surplus; rescueSurplusETH cannot touch tracked liabilities.

---

7. Prioritized Remediation Plan

7.1 P0: Follow the runbook and address frontend Highs

No.	Action	Source
1	Rehearse and use Safe SettleAuctionAtomic.safeBatch(); ban AuthorizeAuction for production	Runbook (agree)
2	Point /admin at atomic Safe settle (not Authorize then Finalize as happy path)	Runbook + FE
3	Auction / NFT / controller on Safe; cancel/pause policy as runbook	Runbook (agree)
4	Frontend: disable production demo query; DemoBar on recovery	New
5	Frontend: stale/degraded write lock on bidding page	New
6	Frontend: demo env cannot ship in production	New

7.2 P1: Same window as launch

No.	Action
7	Bind pending-tx restore to connected account
8	Align WalletConnect metadata URL and explorer with host/chain
9	Degraded-RPC banner when falling back from primary

No.	Action
10	Publish verified Etherscan source for the new renderer contract (rotated 13-14 July; holds latent mint privilege, see C-I4). Done: verified 15 July

7.3 P2: Optional later

No.	Action
11	Hard-disable or remove AuthorizeAuction once unused
12	Typing cleanup / CDN path sanitization

---

8. Residual Risk Statement

By design, and consistent with the runbook:

1. Generation-pipeline roles and NFT owner can mint whenever mintCap exceeds totalMinted.
2. Auction owner can cancel or pause while Active (pause does not freeze the clock).
3. After grace periods, permissionless actors can finalize or drive recovery.
4. Frontend can only fail closed against a lying RPC; it cannot make bad data honest.
5. Host and environment operators control which contract address and RPC users see.

These are accepted trust assumptions, not unfixed Critical bugs.

---

9. Conclusion

Escrow is in good shape, and I checked it twice. On settlement, pause/cancel, and batch hygiene, I agree with AUCTION_RUNBOOK.md: follow that path and the privileged mint window does not need a contract rewrite.

What this engagement adds is independent verification (differential oracle, mainnet fork, and a hand-traced solvency identity), one contract deadline worth respecting (settle within 7 days, per C-M1), and the frontend High fixes (demo lockout, stale bid gating, no demo in production builds). With the runbook enforced and those items done, I'm comfortable with this system proceeding to launch.

Appendix A: Scope and Artifacts

Artifact	Path
Auction contract	contracts/src/PanoramaSeason2Auction.sol
NFT / mint controller	contracts/src/Panorama.sol, PanoramaMintController.sol
Ops runbook (primary ops source)	contracts/AUCTION_RUNBOOK.md
Settlement scripts	contracts/script/SettleAuctionAtomic.s.sol, AuthorizeAuction.s.sol
Differential / fork tests	contracts/test/PanoramaSeason2Auction.differential.t.sol, PanoramaSeason2Auction.mainnet.t.sol
Frontend	frontend/src/

Appendix B: Severity Definitions

Severity	Definition
Critical	Unprivileged theft or permanent loss of user principal; no designed recovery
High	Material harm or severe user deception under realistic conditions
Medium	Significant disruption or fairness issue; often procedure-mitigated
Low	Limited impact, intentional tradeoff, or polish
Agreed / documented	Already covered in project runbook; independently confirmed

---

End of report.